02. Quote 구조 및 파싱

Quote 개요

Quote는 TEE 환경에서 생성되는 암호학적 증명서로, 플랫폼의 신뢰성과 실행 중인 코드의 무결성을 증명합니다. 이 구현체에서는 `Quote::parse()` 함수를 통해 바이너리 데이터를 구조화된 Quote 객체로 변환합니다.

Quote 파싱 프로세스

1. 메인 파싱 함수

```
impl Quote {
pub fn parse(quote: &[u8]) -> Result<Self> {
let mut input = quote;
let quote = Quote::decode(&mut input)?;
Ok(quote)
}
}
```

2. Decode 구현 (`Quote::decode`)

```
impl Decode for Quote {
fn decode<I: Input>(input: &mut I) -> Result<Self, scale::Error> {
```

```rust
let header = Header::decode(input)?;
let report = /* 버전별 Report 파싱 */;
let data = Data::<u32>::decode(input)?;
let auth_data = decode_auth_data(header.version, &mut &data.data[..])?;
Ok(Quote { header, report, auth_data })
}
}
```

Header 구조 분석

Header 필드 (48바이트)

```rust
pub struct Header {
pub version: u16, // 오프셋 0-1: Quote 버전
pub attestation_key_type: u16, // 오프셋 2-3: 증명 키 타입
pub tee_type: u32, // 오프셋 4-7: TEE 타입
pub qe_svn: u16, // 오프셋 8-9: QE 보안 버전
pub pce_svn: u16, // 오프셋 10-11: PCE 보안 버전
pub qe_vendor_id: [u8; 16], // 오프셋 12-27: QE 벤더 ID
pub user_data: [u8; 20], // 오프셋 28-47: 사용자 데이터
}
```

Header 검증 로직

```
impl Header {
pub fn is_sgx(&self) -> bool {
self.tee_type == TEE_TYPE_SGX // 0x00000000
}
}
```

Report 구조별 파싱

1. 버전별 Report 파싱 로직

```
match header.version {
3 => {
// Quote v3: SGX 전용
if header.tee_type != TEE_TYPE_SGX {
return Err(scale::Error::from("invalid tee type"));
}
report = Report::SgxEnclave(EnclaveReport::decode(input)?);
}
4 => {
// Quote v4: SGX/TDX 지원
match header.tee_type {
TEE_TYPE_SGX => report = Report::SgxEnclave(EnclaveReport::decode(input)?),
TEE_TYPE_TDX => report = Report::TD10(TDReport10::decode(input)?),
```

```
_ => return Err(scale::Error::from("Invalid TEE type")),
}
}
5 => {
// Quote v5: Body 구조 포함
let  body = Body::decode(input)?;
match  body.body_type {
BODY_SGX_ENCLAVE_REPORT_TYPE => {
report = Report::SgxEnclave(EnclaveReport::decode(input)?);
}
BODY_TD_REPORT10_TYPE => {
report = Report::TD10(TDReport10::decode(input)?);
}
BODY_TD_REPORT15_TYPE => {
report = Report::TD15(TDReport15::decode(input)?);
}
_ => return Err(scale::Error::from("Unsupported body type")),
}
}
}
```

2. SGX Enclave Report (384바이트)

```rust
pub struct EnclaveReport {
pub cpu_svn: [u8; 16], // CPU 보안 버전 번호
pub misc_select: u32, // 기타 선택 필드
pub reserved1: [u8; 28], // 예약 필드 1
pub attributes: [u8; 16], // Enclave 속성
pub mr_enclave: [u8; 32], // Enclave 측정값
pub reserved2: [u8; 32], // 예약 필드 2
pub mr_signer: [u8; 32], // 서명자 측정값
pub reserved3: [u8; 96], // 예약 필드 3
pub isv_prod_id: u16, // ISV 제품 ID
pub isv_svn: u16, // ISV 보안 버전 번호
pub reserved4: [u8; 60], // 예약 필드 4
pub report_data: [u8; 64], // 리포트 데이터
}
```

3. TDX Report 10 (584바이트)

```rust
pub struct TDReport10 {
pub tee_tcb_svn: [u8; 16], // TEE TCB 보안 버전
pub mr_seam: [u8; 48], // SEAM 측정값
pub mr_signer_seam: [u8; 48], // SEAM 서명자 측정값
pub seam_attributes: [u8; 8], // SEAM 속성
pub td_attributes: [u8; 8], // TD 속성
```

```
pub xfam: [u8; 8], // 확장 기능 가용성 마스크
pub mr_td: [u8; 48], // TD 측정값
pub mr_config_id: [u8; 48], // 구성 ID 측정값
pub mr_owner: [u8; 48], // 소유자 측정값
pub mr_owner_config: [u8; 48], // 소유자 구성 측정값
pub rt_mr0: [u8; 48], // 런타임 측정값 0
pub rt_mr1: [u8; 48], // 런타임 측정값 1
pub rt_mr2: [u8; 48], // 런타임 측정값 2
pub rt_mr3: [u8; 48], // 런타임 측정값 3
pub report_data: [u8; 64], // 리포트 데이터
}
```

4. TDX Report 15 (648바이트)

```
pub struct TDReport15 {
pub base: TDReport10, // 기본 TDReport10 구조
pub tee_tcb_svn2: [u8; 16], // 추가 TEE TCB 보안 버전
pub mr_service_td: [u8; 48], // 서비스 TD 측정값
}
```

TDX 속성 파싱

TD Attributes 구조 (8바이트)

```rust
pub struct TDAttributes {
pub tud: u8, // TD Under Debug 플래그 (비트 7:0)
pub sec: SECFlags, // 보안 속성 (비트 31:8)
pub other: OTHERFlags, // 기타 속성 (비트 63:32)
}
```

속성 파싱 함수

```rust
impl TDAttributes {
pub fn parse(input: [u8; 8]) -> Result<Self, scale::Error> {
let tud = input[0];
// SEC 플래그 추출 (비트 27:8)
let reserved_lower = (((input[3] & 0x0f) as u32) << 16)
| ((input[2] as u32) << 8)
| (input[1] as u32);
let sept_ve_disable = (input[3] & 0x10) != 0; // 비트 28
let pks = (input[3] & 0x40) != 0; // 비트 30
let kl = (input[3] & 0x80) != 0; // 비트 31
// OTHER 플래그 추출 (비트 63:32)
let perfmon = (input[7] & 0x80) != 0; // 비트 63
Ok(TDAttributes { tud, sec: SECFlags { ... }, other: OTHERFlags { ... } })
}
```

```
}
```

AuthData 파싱

버전별 AuthData 구조

```
pub enum AuthData {
V3(AuthDataV3), // Quote v3용
V4(AuthDataV4), // Quote v4용
}
```

AuthDataV3 구조

```
pub struct AuthDataV3 {
pub ecdsa_signature: [u8; 64], // ECDSA 서명
pub ecdsa_attestation_key: [u8; 64], // ECDSA 증명 키
pub qe_report: [u8; 384], // QE 리포트
pub qe_report_signature: [u8; 64], // QE 리포트 서명
pub qe_auth_data: Data<u16>, // QE 인증 데이터
pub certification_data: CertificationData, // 인증서 데이터
}
```

AuthDataV4 구조

```
pub struct AuthDataV4 {
pub ecdsa_signature: [u8; 64], // ECDSA 서명
pub ecdsa_attestation_key: [u8; 64], // ECDSA 증명 키
pub certification_data: CertificationData, // 인증서 데이터
pub qe_report_data: QEReportCertificationData, // QE 리포트 데이터
}
```

인증서 데이터 파싱

CertificationData 구조

```
pub struct CertificationData {
pub cert_type: u16, // 인증서 타입 (5 = PCK 인증서 체인)
pub body: Data<u32>, // 인증서 데이터 본문
}
```

Quote 유틸리티 함수들

1. 인증서 체인 추출

```
impl Quote {
pub fn raw_cert_chain(&self) -> Result<&[u8]> {
let cert_data = match &self.auth_data {
```

```
AuthData::V3(data) => &data.certification_data,
AuthData::V4(data) => &data.qe_report_data.certification_data,
};
if cert_data.cert_type != 5 { // PCK_CERT_CHAIN
bail!("Unsupported cert type: {}", cert_data.cert_type);
}
Ok(&cert_data.body.data)
}
}
```

2. FMSPC 추출

```
impl Quote {
pub fn fmspc(&self) -> Result<Fmspc> {
let raw_cert_chain = self.raw_cert_chain()?;
let certs = utils::extract_certs(raw_cert_chain)?;
let cert = certs.first().ok_or(anyhow!("Invalid certificate"))?;
let extension_section = utils::get_intel_extension(cert)?;
utils::get_fmspc(&extension_section)
}
}
```

3. CA 타입 확인

```
impl Quote {
pub fn ca(&self) -> Result<&'static str> {
let raw_cert_chain = self.raw_cert_chain()?;
let certs = utils::extract_certs(raw_cert_chain)?;
let cert = certs.first().ok_or(anyhow!("Invalid certificate"))?;
let cert_der: Certificate = der::Decode::from_der(cert)?;
let issuer = cert_der.tbs_certificate.issuer.to_string();
if issuer.contains(constants::PROCESSOR_ISSUER) {
return Ok(constants::PROCESSOR_ISSUER_ID); // "processor"
} else if issuer.contains(constants::PLATFORM_ISSUER) {
return Ok(constants::PLATFORM_ISSUER_ID); // "platform"
}
Ok(constants::PROCESSOR_ISSUER_ID)
}
}
```

4. 서명된 데이터 길이 계산

```
impl Quote {
pub fn signed_length(&self) -> usize {
let mut len = match self.report {
Report::SgxEnclave(_) => HEADER_BYTE_LEN + ENCLAVE_REPORT_BYTE_LEN, // 48 + 384
Report::TD10(_) => HEADER_BYTE_LEN + TD_REPORT10_BYTE_LEN, // 48 + 584
```

```
Report::TD15(_) => HEADER_BYTE_LEN + TD_REPORT15_BYTE_LEN, // 48 + 648

};

if  self.header.version == 5 {

len += BODY_BYTE_SIZE; // 6바이트 추가

}

len

}

}
```

이러한 파싱 프로세스를 통해 바이너리 Quote 데이터가 구조화된 Rust 객체로 변환되어 후속 검증 과정에서 활용됩니다.